SDD para Android nativo

OpenSpec, suas skills prioritárias e os complementos de Matt Pocock

Guia prático revisado | 6 de novembro de 2026

Este tutorial organiza o desenvolvimento por pequenas mudanças verificáveis. O OpenSpec mantém a mudança; as 13 skills do seu repositório orientam o trabalho especializado; as skills de Matt Pocock acrescentam métodos quando resolvem uma necessidade concreta.

O exemplo é o aplicativo fictício Notas de Bolso: primeiro notas locais, depois busca e, somente se o produto precisar, sincronização. O processo também serve para outros domínios. Kotlin, Android e interfaces nativas são o alvo; não há KMP nem iOS.

O que você terá ao seguir o guia: documentos com responsabilidades claras, mudanças delimitadas, telas suficientemente detalhadas, tarefas com verificação, revisão independente e um histórico de especificações consistente com o código.

BASE DESTA EDIÇÃO OpenSpec 1.16.0; coleção Matt Pocock no commit 4ae2317ddb22b046e26455e4f9c9b3aaaf62212d; skills locais conforme este workspace, incluindo Peer Review 1.9.0. Há alterações locais ainda não publicadas: instalar do GitHub pode trazer versões anteriores.

Os comandos e prompts são exemplos para você executar no aplicativo que criar. Nenhum aplicativo, teste Android, PR, commit ou release foi produzido para demonstrar os resultados deste tutorial. Evidências de execução devem ser geradas no seu projeto.

Como navegar e o que foi melhorado

Leia a preparação uma vez. Depois siga a sequência de uma change e use os capítulos de referência conforme a tarefa. Os blocos TERMINAL são comandos do shell; os blocos CHAT são mensagens para o agente. Os arquivos de exemplo e prompts estão em inglês; a explicação permanece em português.

Revisão da edição anterior

O histórico registra os pedidos de um roteiro independente do domínio, exemplos de comandos e skills, detalhamento de telas, inclusão de novas skills e critérios de archive. Depois foi solicitada a integração de Matt Pocock, mantendo a prioridade local. A edição anterior tinha 22 páginas; seu texto e gerador foram recuperados para esta revisão.

- Corrigido o nome da skill de sincronização para openspec-sync-specs. O comando canônico de chat equivalente continua /opsx:sync.
- Corrigida a descrição de onboard: é um tutorial guiado pelo fluxo em um repositório, não um gerador genérico de contexto.
- Explicitado que continue cria o próximo artefato; apply retoma implementação. As duas ações têm finalidades diferentes.
- A skill tdd consultada trabalha com red-green em interfaces previamente acordadas; ela deixa refatoração para uma etapa separada. O guia anterior a descrevia genericamente como red-green-refactor.
- Acrescentadas decisões concretas, tarefas rastreáveis, interfaces de teste, atualização de contrato, segunda change e um cenário opcional de background/sync.
- Reforçada a distinção entre disponibilidade, invocação e evidência de uso das skills, sem exigir todas em cada mudança.

Verificação de comandos

A CLI instalada respondeu como 1.16.0. Foram consultados --help de status, validate, archive, instructions, config profile e schema fork. status --all é suportado por esse binário, embora ausente na tabela da documentação consultada; o roteiro principal usa status --change para tornar o alvo explícito.

Os resultados precisam ser obtidos no seu app. Fontes e revisões estão no fim.

Mapa de leitura

Preparação: capítulos 1-10. Primeira entrega Android: 11-21. Busca, diagnóstico e sync opcional: 22-24. Continuidade, novas skills, recursos e checklist: 25-30. Os marcadores do PDF permitem abrir cada capítulo.

Sequência curta, seguindo os detalhes e condições dos capítulos: preparar app e skills; PentaPRO/setup; propose; UI e interfaces de teste; Apply com helpers/TDD; checks; verify e Peer Review; sync-specs; archive; entrega Git/PR. Nem toda change precisa repetir a preparação global.

1. O processo e a autoridade de cada parte

SDD, neste guia, significa tornar a intenção verificável antes e durante a implementação: descrever comportamento, esclarecer decisões materiais, desenvolver uma pequena entrega e comparar o resultado com o contrato. O aprendizado pode exigir voltar à especificação. A ordem não obriga uma cascata rígida.

```
Objetivo e contexto do produto
  -> pequena change OpenSpec
  -> contrato, telas, riscos e verificações
  -> tarefas + skills especializadas
  -> código e evidências
  -> conformidade + Peer Review
  -> correções ou revisão explícita do contrato
  -> sync + archive + entrega Git
  -> próxima change
```

Responsável	Decide ou produz
Você e as regras do projeto	Prioridades, comportamento aceito, escolhas materiais e autoridade de entrega.
PentaPRO	Contexto global e prontidão do próximo incremento, nos cinco pilares.
OpenSpec	Proposta, deltas de comportamento, desenho e tarefas de cada mudança.
Skills locais	UI, segurança, revisão e implementação/testes Android nos respectivos escopos.
Matt Pocock	Métodos opcionais de domínio, desenho de interfaces, testes, diagnóstico e comunicação.
Build, testes e revisão	Evidência sobre o snapshot observado, com limites declarados.

Defina a prioridade no AGENTS.md do app. Instalar coleções não resolve conflitos automaticamente. Uma skill que tenta iniciar outra especificação ou commitar não ganha essa autoridade por estar instalada.

Uma change pequena pode dispensar nova entrevista global. Uma alteração sem tela dispensa UI Design. Um app sem trabalho durável em segundo plano dispensa Android Background. Eficiência vem dessa seleção, e não de uma sequência obrigatória com todos os nomes.

2. Uma fonte principal para cada informação

Esta divisão é uma convenção de integração recomendada para o seu conjunto de skills. Registre-a no projeto; o OpenSpec não a cria sozinho.

Registro	Informação principal e ligações
docs/product.md	Objetivos, público, escopo e políticas globais; aponta para capabilities.
docs/proof.md	Estratégia, critérios de qualidade e índice de evidências; não duplica logs.
docs/project.md	Arquitetura vigente, restrições e resumo/link das decisões duradouras.
docs/production.md	Distribuição, ambientes, migração, operação e recuperação.
docs/process.md	Autoridade, fluxo, prontidão, mudança ativa e links para suas tarefas.
docs/ui-design.md	Contrato cumulativo de telas, estados, interação, acessibilidade e visual.
openspec/specs/	Comportamento consolidado das capabilities, após sincronização aceita.
openspec/changes/<nome>/	Intenção e deltas propostos, design técnico e tasks da mudança.
GLOSSARY.md	Termos do domínio; não contém requisitos nem detalhes de implementação.
docs/adr/	Decisões duradouras difíceis de reverter, com alternativas e consequências.
Tracker, se usado	Coordenação e links para a change; não é outra versão das tasks/specs.

Evite divergência durante uma mudança

Mantenha os requisitos normativos de uma capability nas specs OpenSpec. A UI descreve como o usuário os encontra nas telas e referencia seus IDs. Uma decisão arquitetural pode ter o registro completo em um ADR e apenas um resumo em docs/project.md.

Quando a change propuser um contrato diferente, marque-o como proposto. Atualize os documentos globais afetados de forma coordenada e preserve a diferença entre intenção futura e comportamento vigente. A spec consolidada pode ser atualizada antes do merge Git, dentro da branch da mudança; isso não prova que o comportamento já está em produção.

Não transforme docs/process.md ou o tracker em uma cópia de tasks.md. Links e IDs são suficientes para manter rastreabilidade.

3. Preparar as ferramentas e instalar as skills

Crie o projeto Android nativo com um template compatível do seu ambiente Android. Confirme JDK, SDK, Gradle/AGP, Kotlin e Compose selecionados pelo projeto; não copie números de versão sem verificar sua compatibilidade. Execute o build inicial e registre a linha de base.

TERMINAL - dentro da raiz do novo app

```
node --version
npm install -g @fission-ai/openspec@1.16.0
openspec --version
openspec init --tools codex
```

O npm do OpenSpec consultado exige Node >=22.12.0. A Skills CLI consultada exige Node >=22.20: para usar ambos, atenda ao requisito mais alto. Prefira uma versão suportada pelas ferramentas do seu ambiente. Não execute init no repositório de skills: o destino é o app.

TERMINAL - escolher a coleção local revisada

```
npx skills@latest add /home/dever/workspace/teknolista/skills \
  -a codex
```

Selecione as 13 skills locais. A fonte local inclui alterações ainda não publicadas. A alternativa pública é teknolista/skills como fonte, depois de conferir suas versões. Verifique os diretórios instalados em .agents/skills/ e os SKILL.md, e guarde a origem/revisão usada. Atualizações podem ser cópias ou links conforme o instalador: confira o resultado antes de editar arquivos instalados.

TERMINAL - Matt Pocock como cópia selecionável

```
npx skills@latest add mattpocock/skills -a codex
```

Comece com setup-matt-pocock-skills, domain-modeling, codebase-design, tdd e diagnosing-bugs. Acrescente grill-with-docs, pr, handoff ou retro quando necessários. Um plugin é alternativa documentada pelo autor; escolha plugin ou cópia editável para esta coleção, evitando duplicação. Não é necessário instalar o catálogo inteiro.

Preserve licenças e avisos aplicáveis. A origem Android/PR Quiz deste projeto é atribuída a Philipp Lackner/PL Coding; a licença MIT do repositório não relicencia esse material. Consulte THIRD-PARTY-NOTICES.md da fonte ao distribuir skills.

4. Invocação, perfis e conferência da instalação

O terminal cria estrutura e consulta estado. O agente interpreta contratos, escreve artefatos e implementa. Não digite uma chamada $skill no shell: ali $ tem significado de expansão de variável.

Ação no chat	Codex	Forma canônica OpenSpec
Explorar	$openspec-explore	/opsx:explore
Planejar mudança	$openspec-propose	/opsx:propose
Implementar/retomar código	$openspec-apply-change	/opsx:apply
Revisar artefatos existentes	$openspec-update-change	/opsx:update
Conferir conformidade	$openspec-verify-change	/opsx:verify
Consolidar deltas	$openspec-sync-specs	/opsx:sync
Arquivar	$openspec-archive-change	/opsx:archive

Na integração consultada, Codex usa $nome-da-skill. Outros agentes usam convenções próprias; leia as instruções geradas. Uma menção seleciona uma skill, mas não fornece SDK, credenciais, serviços ou acesso de execução ausentes. Confirme no relato do agente quais instruções ele realmente conseguiu carregar.

TERMINAL - habilitar a verificação opcional

```
openspec config profile
# Selecione os workflows core e acrescente verify.
# Para planejamento incremental, acrescente new e continue.
openspec update --dry-run --json
openspec update
openspec list
openspec list --specs
```

O perfil é configuração do usuário/máquina. update aplica a seleção ao projeto e pode regenerar arquivos de integração; examine a prévia e o diff. Os workflows core consultados são explore, propose, apply, update, sync e archive. verify é opcional e precisa estar habilitado para ser invocado.

Não edite permanentemente os arquivos gerados de OpenSpec para estabelecer a política local. Mantenha essa política em AGENTS.md e openspec/config.yaml, pois uma atualização pode regenerar as integrações.

5. Política de convivência no AGENTS.md

Adapte o AGENTS-template.md do seu repositório ao app. Preserve instruções existentes. O trecho abaixo explicita as decisões de integração; complete os comandos de build e os caminhos reais antes de usar.

ARQUIVO - AGENTS.md, trecho em inglês

```
## Development workflow
This is a native Android/Kotlin JVM application. Do not add
KMP source sets, iOS targets or multiplatform dependencies.

OpenSpec owns each change's proposal, behavior deltas, design
and task list. Use a named change for planned behavior changes.
Project-wide context belongs to the five docs/ PentaPRO pillars.
UI contracts belong to docs/ui-design.md.
GLOSSARY.md defines terms; docs/adr/ records durable decisions.
Link these records instead of duplicating normative content.

Prefer our installed skills for their respective tasks:
penta-pro, ui-design, cybersec, peer-review and the native
Android implementation/testing helpers. pr-quiz is opt-in.

Matt Pocock skills are optional methods at a specific task.
Use tdd at agreed interfaces with our Android test helpers.
Use codebase-design for interface reasoning when useful.
Do not use to-spec, implement or implement-spec as a second
orchestrator for an OpenSpec change. Do not run a second review
workflow for the same purpose unless explicitly requested.

For every material handoff record the change, source snapshot,
contracts, current task, actual checks, gaps and next action.
Never report a check as passed unless matching evidence exists.
Write development artifacts in English; preserve UI localization.
```

Regras que precisam de fatos do app

Defina no mesmo guia onde ficam os módulos, quais variantes são suportadas, os comandos reais de testes e lint, os ambientes isolados e as condições de commit/PR/release. Uma tarefa de implementação já autorizada deve prosseguir; a geração de uma especificação, por si só, não autoriza implementação ou publicação.

Se uma skill externa conflitar com uma decisão do app, siga a autoridade do projeto e explique o conflito. Quando o fluxo externo perder sua finalidade com a adaptação, use a skill local adequada em vez de fingir que executou integralmente a externa.

6. Contexto OpenSpec pronto para adaptar

Edite o config criado por init; não sobrescreva campos específicos já existentes. O contexto abaixo é curto para ser carregado repetidamente. Os caminhos levam aos detalhes.

ARQUIVO - openspec/config.yaml

```
schema: spec-driven
context: |
  Native Android/Kotlin JVM app. No KMP or iOS targets.
  Development artifacts are English; product text is localized.
  Read AGENTS.md for skill routing and document ownership.
  Reuse accepted decisions in the five PentaPRO docs/ pillars.
  UI contracts: docs/ui-design.md. Terms: GLOSSARY.md.
  OpenSpec owns behavior changes and their implementation tasks.
  Prefer our local skills; use Matt Pocock methods selectively.
rules:
  proposal:
    - Link the product objective and existing capabilities.
    - State scope, exclusions and unresolved material decisions.
  specs:
    - Write observable requirements and concrete scenarios.
    - Preserve unaffected scenarios when modifying requirements.
    - Link UI contract IDs; keep behavior explicit in the spec.
  design:
    - Explain data ownership, lifecycle and expected failures.
    - Cover security, compatibility and migration when affected.
  tasks:
    - Name relevant skills and observable completion checks.
    - Include tests and document updates beside the work.
operations:
  apply:
    guidance:
      - Load only the contracts and skills needed for each task.
      - Record executed checks, failures and unavailable evidence.
  archive:
    guidance:
      - Reconcile spec deltas and affected document references.
```

context, rules e operations são orientação para o agente, não executores automáticos nem gates de CI. Não há despacho garantido de skills por colocar seus nomes no YAML. Os requisitos específicos da mudança devem ficar nos artefatos da change; não acrescente todo o produto a este contexto.

7. Iniciar o produto uma vez e configurar Matt

CHAT - contexto global com a skill prioritária

```
$penta-pro
Create the five pillar documents for Notas de Bolso.
First increment: native Android notes stored locally, without
accounts, remote sync, attachments or sharing. Inspect the app
and reuse settled decisions. Ask only consequential open questions.
Link detailed capability behavior to OpenSpec and screen contracts
to docs/ui-design.md. Label assumptions and pending evidence.
This request is planning only; do not implement the app yet.
```

Produza contexto suficiente para o primeiro incremento, mantendo os cinco pilares. Não detalhe todo o produto futuro antes de começar. Com decisões aceitas, peça UI Design para a primeira jornada e Cybersecurity para os dados, superfícies e controles relevantes.

CHAT - configuração externa, uma vez por app

```
$setup-matt-pocock-skills
Configure this existing app without replacing its AGENTS.md.
Keep OpenSpec as the only change/spec/task workflow.
Use local Markdown for coordination for now, not remote issues.
Use root GLOSSARY.md and docs/adr/ for supplementary domain docs.
Present the concrete setup before writing. Preserve local skill
priority and existing document ownership. Do not change Git remotes.
```

O setup consultado explora o repo, propõe tracker e vocabulário de triagem e prepara docs/agents/*.md. Revise os caminhos e o bloco adicionado ao guia existente. Caso escolha GitHub/GitLab, ele também envolve acesso ao repo e pode criar labels; configure isso somente quando fizer parte do trabalho desejado. Tracker local evita esse requisito inicial.

Domínio sem entrevista duplicada

Use domain-modeling quando estiver definindo termos ou decisões. Use grill-with-docs quando faltar uma entrevista focalizada: ela combina grilling e domain-modeling. Passe as respostas já resolvidas pelo PentaPRO. Glossário não é spec; ADR só se justifica por uma decisão duradoura com trade-off real. Nem toda escolha merece outro arquivo.

8. As 13 skills locais no momento certo

Disponibilize a coleção, mas carregue apenas o necessário para o resultado pedido. Não crie bibliotecas, módulos ou funcionalidades para justificar uma skill.

Skill	Gatilho concreto	Saída / limite
penta-pro	Produto, restrições ou prontidão materialmente indefinidos.	Cinco pilares; não reinicia discovery de toda edição.
ui-design	Tela, fluxo, acessibilidade, crítica visual ou UI autorizada.	Contrato UI e implementação/avaliação no modo pedido.
cybersec	Ameaças, mudança de risco, auditoria ou hardening solicitado.	Controles, evidências e riscos; um único assessment.
peer-review	Incremento implementado, testes ou re-review.	Avaliação read-only com achados sustentados.
android-module-structure	Módulos, Gradle ou configuração de build.	Implementação dos limites e configuração aprovados.
android-data-layer	Persistência, transporte, cache, repositório ou sync.	Código de dados e suas fronteiras reais.
android-error-handling	Falhas esperadas e seu mapeamento.	Resultados e recuperação; interação de erro pertence à UI.
android-di-koin	Grafo e lifetime de dependências em projeto com Koin.	Wiring; não exige migrar outro DI para Koin.
android-background	Trabalho durável e adiável com necessidade real.	WorkManager, constraints, retry e cancelamento.
android-unit-testing	Autoria/execução de testes de lógica isolada.	Testes e evidência local; não é revisão independente.
android-integration-testing	Banco, transporte, DI e outras fronteiras não visuais.	Teste na menor infraestrutura que exerce a fronteira.
android-ui-testing	Automação de interação no Android.	Testes de fluxo/semântica; não comprova sozinho qualidade visual.
pr-quiz	Pedido explícito de exercício sobre um PR.	Compreensão; não é revisão nem autorização de merge.

As antigas skills Android de Compose, componentes, MVI e navegação estão dentro de UI Design. Segurança Android está em Cybersecurity. Não instale novamente esses nomes como workflows concorrentes.

No fim de uma tarefa, peça ao agente para declarar quais skills foram carregadas e em que trabalho ajudaram. Isso melhora auditabilidade; uma declaração sem execução/evidência não comprova eficácia.

9. Complementos Matt Pocock e seus limites

Skill externa	Uso útil neste roteiro	Como preservar a prioridade local
setup-matt-pocock-skills	Configurar tracker e documentos auxiliares.	Conferir o bloco de integração uma vez por app.
domain-modeling / grill-with-docs	Resolver linguagem e decisões ainda abertas.	Reutilizar discovery; manter requisitos no OpenSpec.
codebase-design	Desenhar interfaces e localizar seams de teste.	Referência de raciocínio; não obriga novos módulos Gradle.
tdd	Test-first em interfaces acordadas.	Método dentro de Apply; helpers locais implementam testes Android.
diagnosing-bugs	Reproduzir e investigar bug difícil.	Ambiente isolado; correção e contrato continuam na change.
pr	Redigir corpo de PR com mudança, evidência e risco.	Não publica PR nem autoriza merge só por escrever o texto.
handoff	Transferir trabalho para outra sessão.	Apontar para change/snapshot/checks; não criar outro plano.
retro	Propor melhorias de ambiente a partir de uma sessão.	Melhorias de CI/instruções são trabalho separado e revisável.

Os dois cuidados mais importantes

tdd pede acordo prévio sobre as interfaces a testar, com o que cada uma captura e deixa de capturar. Faça esse acordo no planejamento e reutilize-o durante Apply. A skill atual separa refatoração do ciclo red-green. Planeje eventual refatoração como trabalho autorizado e revalidado, sem disparar code-review externo automaticamente.

prototype externo tem saídas orientadas à web e prevê preservar o experimento em uma branch descartável. Para telas Android, prefira UI Design e seus protótipos locais, ou um experimento Compose isolado. Use prototype externo somente para uma pergunta de interação/estado que esse formato possa responder e com os efeitos Git explicitamente desejados. Um HTML não valida IME, lifecycle ou TalkBack nativos.

research externo prevê delegação em background e uma nota com fontes primárias. Use-o quando a ferramenta permitir essa delegação e ela fizer parte do escopo. Sem essa capacidade, faça a investigação permitida diretamente e declare a adaptação; não alegue um subagente inexistente.

10. O que não deve virar outro pipeline

O objetivo é combinar métodos dentro da change, preservando um único lugar para tarefas e comportamento. O catálogo externo inteiro não é uma lista obrigatória.

Possível sobreposição	Convenção recomendada
to-spec versus proposta OpenSpec	Use OpenSpec para esta mudança. to-spec pode criar/publicar uma spec no tracker; não mantenha as duas como normativas.
to-tickets versus tasks.md	Só quando coordenação externa exigir; issues contêm resumo, dependência e link, sem copiar todo o aceite.
implement / implement-spec versus Apply	Use Apply. implement consultada chama TDD, revisão externa e commit; isso conflita com o orquestrador e o controle de entrega escolhidos.
code-review versus peer-review	Peer Review é a primeira escolha. Não inicie duas revisões equivalentes como ritual. Uma segunda lente precisa de escopo e motivo distintos.
wayfinder versus planejamento global	Reservar a programas grandes; cada item executável deve apontar para uma change, sem um segundo backlog normativo.
improve-codebase-architecture versus feature ativa	Avaliação separada; propostas aceitas viram changes explícitas.

triage organiza trabalho recebido; não torna a issue uma spec aceita. ask-matt ajuda a escolher métodos externos; não substitui o roteamento local. teach, wait-what e to-questionnaire ajudam a aprender ou esclarecer decisões, quando esse for o pedido. writing-for-agents pode apoiar manutenção de instruções, sem entrar no fluxo de toda feature.

CHAT - uma pergunta de domínio, quando realmente aberta

```
$grill-with-docs
Clarify only what "saved note" means in this product.
Read the accepted PentaPRO decisions first. Do not repeat them.
Record settled vocabulary in GLOSSARY.md; propose an ADR only
for a durable trade-off. OpenSpec remains the behavior contract.
Do not generate another spec, implementation plan or tickets.
```

Se não houver dúvida material, pule essa chamada. Menos entrevistas repetidas deixam mais tempo para cenários, implementação e observação do resultado.

11. Exemplo: planejar três incrementos úteis

Notas de Bolso começa com armazenamento local. A escolha didática usa Compose e Room, com ViewModel para estado de tela. Koin só entra se for a solução escolhida no app; múltiplos módulos só se a estrutura justificar. Essas escolhas são do exemplo, não requisitos universais deste roteiro.

Change	Resultado vertical	Principais skills
notes-create-edit	Criar, abrir, editar e reencontrar nota após reiniciar o app.	UI, dados, erros, unit/integration/UI tests; revisão.
notes-search	Encontrar notas por substring literal do título, com regra explícita.	UI, dados e testes; TDD/diagnóstico opcionais.
notes-sync	Se o produto exigir: sincronizar notas por conta com política de conflito.	PentaPRO para decisões novas; security, dados, background, DI e testes pertinentes.

Não implemente sync para usar Background. O primeiro incremento já deve entregar valor e comprovar persistência. Uma feature de notas pode sobreviver a muitas changes; ela não precisa de uma change aberta para sempre.

Decisões aceitas para o primeiro exemplo

- Criar exige título que não seja apenas espaço; o corpo pode estar vazio. Exibir erros de validação sem perder o texto.
- Salvar com sucesso significa que o armazenamento local confirmou a operação. Só então voltar à lista.
- Editar altera a mesma identidade da nota. Dois toques ou retry não devem criar outra nota.
- Conteúdo já salvo sobrevive ao encerramento do processo e reabertura. Rascunho não salvo sobrevive à rotação; este incremento não promete recuperá-lo após morte do processo.
- Back com alterações não salvas pede confirmação para descartar; cancelar a confirmação mantém edição.
- Falha de storage mantém editor e texto disponíveis; retry é explícito. O app local não exige rede nem apresenta erro de rede para salvar localmente.

Essas decisões deixam o exemplo implementável. Para outro app, substitua-as por políticas próprias. Se salvar rascunhos após morte do processo for essencial, especifique essa garantia adicional e sua persistência, em vez de presumir que ViewModel a fornece.

12. Descobrir e propor notes-create-edit

TERMINAL - procurar trabalho ou capacidade já existente

```
git status --short
openspec list
openspec list --specs
```

Leia os documentos globais e os SKILL.md realmente instalados. Se houver uma capability equivalente, altere-a; não crie outra só com nome diferente. Não misture mudanças locais de outro trabalho.

CHAT - explorar somente se uma decisão continuar aberta

```
$openspec-explore
For notes-create-edit, inspect the native app and accepted docs.
Compare local persistence and screen-state recovery options.
Keep saved-note persistence separate from unsaved-draft recovery.
List only consequential unresolved choices. Do not implement code.
```

CHAT - criar os artefatos da primeira change

```
$openspec-propose notes-create-edit
Plan the accepted local create/edit journey in docs/product.md
and docs/ui-design.md. Use a notes capability; reuse it if present.
Apply the decisions listed for this increment, including storage
failure, duplicate submission, Back and saved data after restart.
Exclude accounts, sync, attachments and sharing.
Produce proposal, behavior deltas, technical design and tasks.
Identify the applicable local skills and agreed test interfaces.
Keep material unknowns explicit. This request is planning only.
```

Saída esperada, não evidência de execução

```
openspec/changes/notes-create-edit/
  .openspec.yaml
  proposal.md
  specs/notes/spec.md
  design.md
  tasks.md
```

O CLI new change cria a estrutura/metadata; é o agente, orientado por status/instructions, que escreve os artefatos. Propose não é geração automática de um app e não é aprovação da proposta. Leia e ajuste o conteúdo antes de autorizar Apply.

13. Exemplo de delta comportamental

ARQUIVO - changes/notes-create-edit/specs/notes/spec.md

```
## Purpose
Keep personal notes locally and make acknowledged saves available
to the user after the Android application is restarted.

## ADDED Requirements

### Requirement: Save a local note
The app SHALL save a note with a nonblank title without requiring
a network connection and SHALL acknowledge only committed saves.

#### Scenario: Create and reopen
- **WHEN** the user saves a valid new note while offline
- **THEN** the list shows the saved title and body
- **AND** the same saved content is available after app restart

#### Scenario: Reject a blank title
- **WHEN** the user saves a title containing only whitespace
- **THEN** a title validation error is shown
- **AND** no note is stored and the entered content remains

#### Scenario: Recover from a storage failure
- **WHEN** local persistence fails before save acknowledgement
- **THEN** the editor retains the entered content and offers retry
- **AND** retrying the same creation does not duplicate the note

### Requirement: Edit the same note
The app SHALL apply acknowledged edits to the selected note
without creating another note, including repeated Save actions.

#### Scenario: Repeated save
- **GIVEN** an existing note with a known identity
- **WHEN** the user edits it and invokes Save repeatedly
- **THEN** that identity has the new values and no duplicate exists

### Requirement: Preserve or discard unsaved edits
The app SHALL preserve unsaved input across configuration changes
and SHALL request confirmation before discarding dirty input.

#### Scenario: Cancel leaving the editor
- **WHEN** the user leaves a dirty editor and cancels discard
- **THEN** the editor remains open with the entered content

#### Scenario: Recreate on rotation
- **WHEN** Android recreates the screen for a configuration change
- **THEN** the editor still contains the unsaved title and body
```

O exemplo cobre as decisões centrais desta entrega. O design e os testes ainda devem detalhar os caminhos necessários para comprovar cada garantia; confirme que a change gerada não perdeu nenhum cenário aceito.

Use ADDED para requisitos novos, MODIFIED para o requisito existente completo, REMOVED para remoção com motivo/migração e RENAMED para mudança de nome. Uma edição em MODIFIED não deve apagar cenários que continuam válidos. Purpose aplica-se à capability nova; respeite o template retornado pela versão instalada.

14. Telas com comportamento implementável

CHAT - a skill local é dona do contrato de UI

```
$ui-design
Document NOTE_LIST and NOTE_EDIT in docs/ui-design.md for
notes-create-edit. Read its delta and the accepted global docs.
Define states, actions, Back, keyboard, accessibility and recovery.
Keep observable note behavior normative in the OpenSpec spec.
Label proposed versus accepted choices. Do not implement yet.
```

Campo	NOTE_LIST	NOTE_EDIT
Entrada/saída	Abertura do app; selecionar nota abre edição; Nova nota inicia criação.	Nova ou identidade existente; salvar confirmado retorna à lista.
Conteúdo	Título e prévia, ação Nova nota; ordem de atualização mais recente primeiro, com desempate estável definido no design.	Título, corpo, Salvar; identidade existente preservada.
Estados	Carregando, lista, vazio, erro de leitura com retry.	Carregando nota, editando, validação, salvando, erro recuperável.
Erro versus vazio	Erro de leitura não vira "nenhuma nota".	Falha mantém texto; não navegar como se salvo.
Back/teclado	Navegação e IME conforme Android e tarefa corrente.	Fechar IME respeita comportamento nativo; ao sair com dirty state, confirmar descarte.
Recriação	Reabrir itens efetivamente salvos.	Rotação preserva rascunho; não confundir com morte do processo.
Acessibilidade	Ação Nova nota rotulada, lista com semântica e ordem lógica.	Campos com labels, erro identificável, foco e anúncio apropriados.

Definições visuais e evidências

Especifique hierarquia, espaçamento/tokens existentes, strings, estados habilitados, insets e layouts estreitos/largos pertinentes. Escolha casos de texto ampliado, tema e conteúdo que exponham problemas reais. Verifique toque, TalkBack, foco, contraste e teclado no ambiente nativo quando essa conclusão for necessária.

Não trate screenshots ou um protótipo HTML como prova de funcionamento Android. Um protótipo responde uma pergunta; o contrato aceito volta para docs/ui-design.md e a spec afetada. Mantenha propostas fora do trecho que descreve UI já aceita.

15. Desenho e tarefas ligadas à comprovação

design.md explica como o comportamento será entregue. No exemplo, o identificador da nova nota é estável durante Save/retry; a escrita usa a identidade existente; a UI só reconhece sucesso após a confirmação do repositório. Não retenha Activity em dependências de longa duração, nem faça I/O bloqueante na thread principal. Cancelamento continua sendo cancelamento.

ARQUIVO - tasks.md, exemplo resumido

```
## 1. Contracts and test interfaces
- [ ] 1.1 Reconcile note scenarios and NOTE_LIST/NOTE_EDIT.
  Skills: ui-design. Check: success, error, Back and recovery agree.
- [ ] 1.2 Record agreed test interfaces in design.md.
  Skills: codebase-design (optional), android-unit-testing.

## 2. Local persistence and errors
- [ ] 2.1 Implement save/edit with stable note identity and retry.
  Skills: android-data-layer, android-error-handling.
  Check: no duplicate after repeated save; failed save keeps input.
- [ ] 2.2 Exercise the real Room adapter with isolated storage.
  Skill: android-integration-testing. Check: durable reopen/edit.

## 3. Native journey
- [ ] 3.1 Implement Compose list/editor and accepted UI states.
  Skill: ui-design. Check: blank title, error, Back and rotation.
- [ ] 3.2 Automate critical interactions and review accessibility.
  Skills: android-ui-testing; ui-design for visual/native evidence.

## 4. Verification and reconciliation
- [ ] 4.1 Run applicable checks and record actual results.
- [ ] 4.2 Obtain conformity and peer-review reports; resolve gaps.
- [ ] 4.3 Reconcile affected docs and prepare spec sync/archive.
```

Cada tarefa deve ser pequena o bastante para ter resultado observável, mas pode conter implementação e teste juntos. Não deixe todos os testes para a etapa 4. As linhas de Skill orientam o agente; não executam um dispatcher automático.

Módulos/Gradle ou Koin entram em tarefas específicas se houver mudanças concretas nessas áreas. Omitir um helper não aplicável é correto. Para marcar uma tarefa, declare o trabalho implementado e as verificações previstas; se faltar evidência obrigatória, mantenha visível a pendência correspondente.

16. Acordar as interfaces de teste e aplicar TDD

Na terminologia de Matt, seam é a interface onde o comportamento pode ser observado/substituído sem o teste depender dos detalhes internos. O significado não é "um módulo Gradle por teste".

Interface acordada	O que deve detectar	O que não estabelece
Estado/ações públicas do editor	Título inválido, falha/retry, identidade estável e conteúdo preservado.	Funcionamento real de Room ou do sistema Android.
Repositório com adaptador Room real	Save/edit e reabertura com banco de teste; integridade da identidade.	Navegação, IME, aparência ou acessibilidade.
Jornada Compose instrumentada	Nova nota, salvar, editar, confirmação de Back e erro apresentado.	Todas as variantes de lifecycle ou recuperação após morte do processo.

CHAT - confirmação única antes de escrever os testes

```
Use $codebase-design only to assess these proposed test interfaces.
For each, state the behavior caught and the coverage left open.
Record the agreed interfaces in notes-create-edit/design.md.
Ask me to confirm unresolved interface choices before using $tdd.
Preserve our existing Android architecture and test framework.
```

O acordo pode ser feito no mesmo planejamento da change. Reutilize-o enquanto continuar válido, sem pedir confirmação repetida para o mesmo escopo.

CHAT - começar a implementação autorizada

```
$openspec-apply-change notes-create-edit
Implement the agreed tasks. Use $tdd at the confirmed interfaces.
Use our android-unit-testing and android-integration-testing
skills to implement the Android tests, and ui-design for the UI.
Use android-data-layer and android-error-handling for data work.
Keep Apply as task owner. Do not start implement/implement-spec
or external code-review. Record actual red and green evidence.
Do not commit, publish a PR or deploy in this implementation step.
```

Exemplo de um ciclo: testar "falha mantém texto"; observar falha pelo motivo esperado; implementar o mínimo para passar; verificar o cenário. Próximo ciclo: retry usa a mesma identidade. Red por erro de importação ou fixture quebrada não demonstra que o teste detecta a regressão pretendida. Refatoração necessária é uma tarefa autorizada, com revalidação, respeitando o processo local.

17. Verificações Android com evidência real

Descubra os nomes e a localização das tasks no app gerado. Os comandos abaixo são exemplos comuns, não garantias de que todo projeto tenha módulo :app ou variante Debug.

TERMINAL - descobrir, executar e registrar

```
./gradlew tasks --all
./gradlew :app:testDebugUnitTest
./gradlew :app:lintDebug
./gradlew :app:assembleDebug
# Com emulador/dispositivo e testes configurados:
./gradlew :app:connectedDebugAndroidTest
openspec validate notes-create-edit --type change --strict \
  --no-interactive
```

Rode também os checks exigidos pelo app/CI. Se uma mudança afeta minificação, permissões ou configuração de release, complemente com a variante apropriada. Não use apenas Debug para concluir que o pacote distribuído está correto.

Evidência necessária	Como obtê-la no exemplo
Retry/identidade/validação	Testes do contrato público do editor e do repositório com fakes controlados.
Persistência real	Adaptador Room com armazenamento isolado; fechar/reabrir acesso e confirmar conteúdo/ID.
Jornada do usuário	Testes instrumentados nos estados e interações aceitos.
Saved note após processo encerrar	Procedimento isolado com banco preservado e reabertura do app; registrar ambiente.
Rascunho após rotação	Recriar tela/configuração e verificar texto; não relatar como teste de morte do processo.
Qualidade visual/acessível	Inspeção nativa nos cenários escolhidos, com tamanho de texto, TalkBack e teclado pertinentes.

Registre comando, snapshot, variante, ambiente, seleção e resultado: Passed, Failed ou Not run com motivo. Um processo com exit 0 e zero testes não comprova nada sobre o cenário. Diferencie falha herdada, falha introduzida e indisponibilidade do ambiente.

Não invente logs "de exemplo" como resultados reais. O PDF oferece critérios e comandos; você preencherá o registro com o que observar no app.

18. Conformidade, revisão e segurança

CHAT - OpenSpec compara o resultado com os artefatos

```
$openspec-verify-change notes-create-edit
Compare the current implementation with its requirements,
scenarios, design and tasks. Report gaps and evidence read-only.
Distinguish source inspection from tests actually executed.
```

CHAT - fresh reviewer, com o material necessário

```
$peer-review
Review notes-create-edit at the specified source snapshot.
Read AGENTS.md, the change, UI contract and relevant tests.
Reconcile the full file list, deletions, config and dependencies.
Inspect required check evidence for this exact snapshot.
Trace save/retry identity, storage acknowledgement and lifecycle.
Report substantiated defects, test-protection gaps and limits.
Do not edit source or Git state. Return findings and coverage.
```

Use uma sessão independente com acesso ao código e aos contratos; não entregue apenas o raciocínio do implementador. Um modelo de outro fornecedor é opcional. Se for autorrevisão, declare essa limitação. Use os templates locais de relatório/handoff para entregas maiores.

Peer Review 1.9.0 combina evidência automática com análise semântica. Migrations, autenticação/configuração, dependências e enfraquecimento de testes/CI pedem profundidade adicional. Um teste obsoleto pode ser retirado com justificativa; apagar uma proteção para obter verde exige investigação. Um bug demonstrado não vira "estilo" apenas porque o implementador discordou.

CHAT - segurança com escopo concreto

```
$cybersec
Assess the local-note data flow: storage, backup, logging and
Android exported surfaces. Use the accepted privacy policy.
Report controls, negative checks, unavailable evidence and
residual risks. Do not use production data or change the app.
```

Aplique esse assessment quando o escopo/pedido o exigir. Corrija achados confirmados em uma etapa autorizada e repita os checks afetados. Validate, verify, testes e revisão respondem perguntas diferentes; nenhum resultado verde substitui todos os demais.

19. Quando o plano ou o contrato mudarem

Se o código estiver errado, corrija contra o contrato aceito. Se a necessidade mudou, revise o contrato e suas consequências explicitamente. Não mude a spec apenas para combinar com uma implementação falha.

CHAT - revisão de artefatos existentes

```
$openspec-update-change notes-create-edit
The accepted product decision now requires recoverable drafts
after process death, not only rotation. Show the proposed changes
to requirements, design and tasks, with affected UI and tests.
Keep unrelated decisions and existing completed evidence intact.
Do not implement code as part of updating the planning artifacts.
```

O workflow update consultado apresenta revisões para confirmação por artefato. Ele edita planejamento dentro da change; documentos cumulativos externos, como UI, precisam ser reconciliados com suas skills donas. Uma mudança de intenção muito grande pode justificar outra change.

Evite uma referência circular de verdade

1. Registre a nova decisão aceita e o que ela substitui.
2. Atualize o requisito inteiro em MODIFIED, preservando cenários ainda válidos.
3. Reavalie o mecanismo técnico e o contrato de UI. Draft persistido tem implicações de storage, privacidade e limpeza que ViewModel sozinho não resolve.
4. Acrescente as tarefas e verificações novas. Não relabele evidência antiga como se já tivesse testado a nova garantia.
5. Retome Apply e depois verify/review nos caminhos afetados.

Mudança puramente interna

Quando não houver delta de comportamento, uma change pode declarar skip_specs: true em .openspec.yaml, conforme o schema/CLI consultado. Justifique no proposal e mantenha design/tarefas/checks pertinentes. Não use essa opção para esconder mudança real de contrato, permissão, compatibilidade ou dados.

Se um artefato ainda não foi criado, new/continue ou as instruções retornadas pelo CLI ajudam a produzi-lo; update não é o comando para avançar automaticamente por todos os artefatos faltantes.

20. Sincronizar e arquivar com intenção

OpenSpec arquiva changes, não a ideia de uma feature. A capability continua nas specs enquanto o comportamento for suportado. Uma feature longa é entregue por várias changes pequenas.

Antes do archive deste exemplo

- Os cenários aceitos estão implementados e as tarefas exigidas concluídas; o que foi transferido tem destino e decisão explícitos.
- Checks obrigatórios e revisões estão resolvidos no snapshot identificado, com limitações registradas.
- Os deltas e os documentos cumulativos afetados foram reconciliados; o diff de sync foi revisado.
- A equipe definiu se arquiva na branch antes do PR final ou em uma alteração posterior coordenada. Esta edição recomenda arquivar na mesma branch antes do commit final de entrega.

CHAT - caminho principal recomendado

```
$openspec-sync-specs notes-create-edit
Merge only this change's behavior deltas into the main specs.
Preserve unaffected requirements and scenarios. Report the diff
and any conflicts; do not archive or commit in this step.
```

Revise o diff. Então execute:

```
$openspec-archive-change notes-create-edit
Check artifact/task state and pending spec deltas. Archive the
completed change and report what was synced and where it moved.
Do not merge Git branches, publish the app or alter remote rules.
```

Alternativa CLI, não uma segunda etapa obrigatória

```
openspec archive notes-create-edit --yes
openspec validate --all --strict --no-interactive
git diff --stat
```

O comando CLI archive valida, aplica deltas por suas regras e arquiva; a skill faz o workflow guiado pelo agente. Escolha um caminho de archive. --yes remove prompts; não aprova automaticamente a qualidade do app. Evite --no-validate ou --skip-specs para contornar pendências.

Não arquive como "concluída" uma change cancelada apenas para limpar a lista, nem sincronize comportamento não entregue como se estivesse vigente. Registre cancelamento/adiamento e sua destinação conforme a política do projeto; não marque tarefas ficticiamente. Arquivar não faz deploy.

21. Entrega Git, PR e quiz no momento certo

Neste roteiro, código, testes, documentos atualizados, specs consolidadas e histórico da change seguem juntos na mesma entrega. Revise o diff produzido por sync/archive também: ele é parte do conteúdo a publicar.

Sequência de entrega

1. Obter evidência do código e resolver achados. Sincronizar e arquivar na branch da mudança.
2. Conferir o conjunto final de arquivos e repetir os checks afetados por edições posteriores. Um relatório de revisão pertence ao snapshot que examinou.
3. Fazer commit e push conforme a autorização de entrega e as regras do repositório. O ato de especificar ou testar não concede sozinho essa autorização.
4. Criar/atualizar PR com comportamento, evidência e riscos. Fazer merge somente após os gates exigidos.
5. Publicar versão Android seguindo docs/production.md: assinatura, configuração, dados/migrations, distribuição e recuperação pertinentes.

CHAT - usar Matt para redigir o PR

```
$pr
Draft the PR body for the final reviewed diff of notes-create-edit.
Link the archived change and consolidated capability. Summarize
the observable change, actual evidence and remaining limitations.
Explain migration/rollback risks if any. Do not invent before/after
results. Draft only; do not create a remote PR in this step.
```

CHAT - quiz opcional, após o diff estabilizar

```
$pr-quiz
Draft a decision-comprehension quiz for this PR's final snapshot.
Focus on save acknowledgement, identity, retry and lifecycle.
Use the real PR/source content. Do not treat answers as approval.
```

Se quiser publicar o quiz ou instalar o workflow de correção, faça esse pedido explicitamente. O template da skill é inativo até ser instalado. Como o resultado está ligado ao commit, commits posteriores exigem reavaliação; não use um passe antigo para um HEAD novo.

Você pode criar um PR draft antes do fim para colaboração; nesse caso, atualize seu conteúdo e as evidências quando sync, archive ou correções mudarem o snapshot. A ordem recomendada aqui reduz retrabalho no primeiro app.

22. Segunda change: busca sem ambiguidades

Agora suponha notes-create-edit concluída. A busca é uma nova entrega, não outra especificação global do app.

Decisões do exemplo notes-search

Busca somente no título; substring literal; insensível a caixa para as letras ASCII A-Z/a-z; os demais caracteres têm comparação literal nesta primeira versão. Consulta vazia, após trim, retorna todas as notas. Não há pesquisa remota. Preservar a ordem da lista e distinguir lista sem notas, nenhum resultado e erro de leitura. Para outro idioma/domínio, escolha e teste as regras de normalização apropriadas; não deixe "busca inteligente" sem definição.

CHAT - propor e detalhar a tela existente

```
$openspec-propose notes-search
Add local title search with the agreed matching rules.
Reuse the notes capability and NOTE_LIST contract.
Include empty query, no match, literal '%' and '_', clearing the
query, and stale asynchronous results. Exclude remote search.
Keep OpenSpec tasks canonical. Do not use to-spec/to-tickets.
```

Depois use ui-design para acrescentar campo, limpar filtro, foco/IME, feedback e estados no contrato cumulativo NOTE_LIST. Não refaça a tela nem adicione outro documento de UI só porque existe uma nova change.

Casos que devem distinguir comportamento correto

Notas e consulta	Resultado esperado
Título Plan / consulta plan	Encontra, pela regra ASCII acordada.
Título Pão / consulta pao	Não encontra nesta política literal de acento.
Título 100% / consulta %	Encontra por caractere literal, sem wildcard acidental.
Qualquer lista / consulta só espaços	Retorna lista sem filtro.
Consulta A demora; consulta B termina antes	Apenas o resultado da consulta corrente controla a UI.

Se a consulta usa LIKE/FTS/índice, o desenho deve respeitar o contrato de matching e o escape real do mecanismo. Parametrizar SQL impede injeção no valor; não resolve sozinho a semântica de '%' e '_' como wildcards.

23. Diagnóstico e regressão na segunda change

Suponha que notas-search falhe para '%'. O teste do estado de tela com repository fake pode passar, pois não usa a query real. Isso explica onde complementar evidência; não torna o teste unitário inútil.

CHAT - diagnóstico opcional com Matt

```
$diagnosing-bugs
Diagnose notes-search: searching for literal '%' returns notes
without that character. Use the accepted matching contract.
Build a small reproducible check against the actual local query
adapter with synthetic notes. Do not use production data, change
the contract to match the bug, or perform Git history operations.
Keep the fix tracked in the current OpenSpec change.
```

diagnosing-bugs exige construir um sinal de falha antes da investigação aprofundada. Se isso não for viável no ambiente, relate a limitação e escolha uma inspeção autorizada apropriada; não invente uma reprodução ou continue afirmando cumprir integralmente esse workflow.

Autoria dos testes e re-review

Android Integration Testing deve exercer o adaptador real. TDD pode guiar red-green na interface previamente acordada. Corrija o escape/mecanismo da consulta e acrescente o caso literal ao conjunto normal, não apenas a um script temporário.

```
$openspec-apply-change notes-search
Implement the scoped fix and regression protection. Use our data
and integration-testing skills and the agreed tdd interface.
Keep the real query in the regression check; isolate storage.
Record old/new behavior only where actually observed.
```

Depois de testes e conformidade, peça peer-review com foco na correção e nos caminhos adjacentes, então sync/archive. Se esse bug já tivesse escapado para produção, acrescente o ciclo da Peer Review 1.9.0: qual contrato falhou, por que o check anterior não detectou e qual melhoria específica previne repetição.

Uma retro com Matt pode propor melhor navegação, regra de seleção de testes ou um check determinístico. Leia primeiro a infraestrutura existente; não crie uma coleção inteira de gates para um incidente isolado. Propostas de ambiente/CI permanecem distintas da correção urgente e precisam de revisão própria.

24. Terceiro incremento, somente se houver sync

notes-sync ilustra quando as skills restantes passam a ter motivo real. Não é um convite para adicionar um backend ao primeiro app. Sem necessidade de sync, pule este capítulo e deixe Android Background não aplicável.

Antes de propor

PentaPRO deve resolver o que mudar materialmente: conta, fonte de verdade, operação offline, conflitos, logout, exclusão e suporte. Cybersecurity deve considerar isolamento por conta, credenciais, transporte, logs e dados pendentes. O contrato da API e seus testes são pré-requisitos; o app não pode inventar garantias do servidor.

CHAT - planejar com as perguntas certas

```
$openspec-propose notes-sync
Plan account-scoped note synchronization using the approved API.
Specify ownership, conflict rules, idempotency, offline edits,
logout/account switching and deletion before implementation.
Use penta-pro for unresolved product/architecture decisions and
cybersec for threat/control checks. Do not invent server behavior.
Name applicable local data, error, background, DI and test skills.
```

Trabalho concreto	Skill prioritária e verificação
Enviar pendências e reconciliar versões	android-data-layer; teste do protocolo, conflito e falha parcial.
Retry transitório versus falha definitiva	android-error-handling; cancelamento não vira erro recuperável.
Trabalho durável adiável	android-background; constraints, unicidade, idempotência e reexecução.
Injetar dependências do Worker, se Koin já escolhido	android-di-koin; criação Android correta e ownership dos parâmetros.
Alterar módulos ou configuração de build	android-module-structure; build e fronteiras do projeto aprovado.
Estado de sync nas telas	ui-design; offline, pendente, erro, conflito e recuperação aceitos.
Correr logout/troca de conta com uma resposta antiga	Testes de dados/integração e cybersec/peer-review no escopo pertinente.

Não prometa horário exato com WorkManager. Uma checagem de conta antes da requisição não impede resposta antiga de gravar dados após logout; estabeleça a regra e a fronteira de commit por conta. Credenciais e valores privados não devem entrar em specs, fixtures ou relatórios.

25. Continuar em outra sessão ou modelo

Use o mesmo ID da change. O estado persistente é o ponto de partida; o agente deve verificar o código atual e o trabalho ainda pendente, não apenas acreditar em checkboxes ou em um resumo anterior.

TERMINAL - orientação ao retomar

```
git status --short
git rev-parse HEAD
openspec list
openspec status --change notes-search --json
openspec instructions apply --change notes-search --json
```

status descreve artefatos. O fato de todos existirem não implica que tasks estejam executadas ou que o app funcione. instructions devolve orientação e contexto; não é implementação por si só.

Pacote mínimo de contexto

```
Change: notes-search
Target: branch + HEAD + identity of relevant uncommitted files
Contracts: notes capability, NOTE_LIST, agreed matching policy
Current task: real-query literal matching regression
Done: list actual implemented work; link actual evidence
Pending: exact task/check, reason and required environment
Skills: data + integration-testing; tdd interface already agreed
Decisions: accepted choices versus proposals/unresolved questions
Next action: reproduce the focused case, then finish the fix
```

CHAT - retomar implementação

```
$openspec-apply-change notes-search
Resume from the current code and artifacts, not only the summary.
Recheck evidence freshness. Preserve unrelated local changes.
Continue the first incomplete coherent task with its relevant skill.
Report any material conflict before dependent work proceeds.
```

Para modelos com menos contexto, prefira changes menores, entradas concretas e resultados observáveis. Carregue apenas a seção relevante de cada skill. Interrompa geração excessiva de planos paralelos e retome o arquivo canônico. Peça handoff externo apenas quando precisar transferir contexto; ele deve apontar para esses registros.

26. Adicionar uma skill sem duplicar objetivos

Adicionar uma skill é manutenção de instruções do agente, não adicionar uma funcionalidade ao app. Primeiro compare objetivo, gatilho, modo, entradas, autoridade, saídas e evidência com as 13 existentes e as externas selecionadas.

Se a necessidade for "mais exemplos de Room" ou "revisão de performance Android", amplie o helper/owner pertinente. Só crie outro entrypoint para uma tarefa distinta. Uma possibilidade é authoring/execução de benchmarks Android; Peer Review já avalia alegações de medição, mas não é o helper que implementa um harness.

ARQUIVO - exemplo inicial a adaptar, não skill pronta

```
---
name: android-benchmark-authoring
description: Author or run isolated Android benchmarks for a named
  performance question; not general code review or a device audit.
metadata:
  version: "0.1.0"
---
# Android Benchmark Authoring
Inspect the actual app, target variant and available benchmark
toolchain. Establish the user path, correctness check, baseline,
measurement conditions and permitted side effects before running.
Reuse the project contracts. Record artifacts, environment,
variation and limits. Do not equate a microbenchmark with UX.
Use peer-review for read-only assessment of the resulting evidence.
```

Antes de instalá-la, acrescente os procedimentos específicos que a tornem útil, referências atuais, exemplos e casos de fronteira. O trecho não substitui pesquisa do toolkit nem valida a skill.

Atualize README, SKILL-ROUTING, versões, referências e metadados relevantes. Teste ativação e não ativação com tarefas realistas; scripts exigem testes de seu comportamento. Registre os resultados efetivos e a origem/licença de material aproveitado.

Valide com o skill-creator disponível no ambiente. Não hardcode caminhos pessoais nos procedimentos reutilizáveis. Em outro aplicativo, instale a skill completa e confira descoberta; a nova pasta no repositório de origem não aparece automaticamente em todos os projetos.

27. Referência de workflows OpenSpec

Use a ação que resolve a necessidade atual. Os nomes abaixo foram conferidos na documentação e nos nomes de skills do pacote instalado 1.16.0. A disponibilidade depende do perfil e da integração gerada.

Ação canônica	Skill Codex	Momento
explore	openspec-explore	Analisar uma dúvida antes de código; salvar planejamento apenas quando pedido/aceito.
propose	openspec-propose	Gerar o planejamento necessário para uma change.
new	openspec-new-change	Criar o scaffold; expandido.
continue	openspec-continue-change	Gerar um próximo artefato liberado; expandido.
ff	openspec-ff-change	Gerar artefatos necessários em sequência; expandido. Não pula implementação.
apply	openspec-apply-change	Implementar ou retomar tarefas.
update	openspec-update-change	Reconciliar artefatos de planejamento já existentes.
verify	openspec-verify-change	Relatar completude/coerência com a implementação; expandido.
sync	openspec-sync-specs	Mesclar deltas em specs principais sem arquivar.
archive	openspec-archive-change	Encerrar a change com análise de estado/deltas.
bulk-archive	openspec-bulk-archive-change	Tratar várias changes concluídas; conferir conflitos antes.
onboard	openspec-onboard	Tutorial guiado no repositório; usar ambiente descartável para experimentar.

new + continue é útil para revisar uma decisão por vez; propose é o caminho compacto. Não execute ambos para criar duas cópias da mesma mudança. ff apenas acelera a escrita de planejamento, e ainda exige inspeção antes de Apply.

Com várias changes, mantenha contratos compartilhados explícitos. Se duas alteram o mesmo requisito, resolva intenção e combinação antes de sync/bulk archive. Um merge textual sem conflitos não prova compatibilidade semântica. Para começar sozinho, uma change ativa por vez reduz a quantidade de estado que você precisa acompanhar.

28. CLI e recursos avançados sem complexidade precoce

TERMINAL - navegação e instruções

```
openspec list --json
openspec list --specs
openspec show notes-search
openspec status --change notes-search --json
openspec instructions specs --change notes-search --json
openspec instructions apply --change notes-search --json
openspec validate --all --strict --no-interactive
openspec schemas
openspec templates --schema spec-driven
```

Consulte --help de cada comando na instalação usada. instructions/templates orientam o agente; o CLI não escreve sozinho todo o conteúdo de spec e app. Inspecte também mensagens informativas de validação: exit 0 não transforma todos os avisos sobre integração em provas de compatibilidade.

Customização

Comece com spec-driven, context e rules. Não recrie os cinco pilares PentaPRO como cópias de artefatos em toda change. Se um novo artefato for realmente necessário, avalie um schema local e sua manutenção:

```
openspec schema fork spec-driven android-team
openspec schema validate android-team
openspec schema which android-team
```

Use schema init para começar um esquema próprio quando houver motivo; preserve dependências e instruções coerentes. Uma customização não pode declarar pronto um trabalho que ainda precisa de evidência.

Multi-repositório e utilidades

Stores separam planejamento em um repositório próprio e são apresentados como beta na fonte consultada. store setup/register/list/doctor ajudam a administrá-los; worksets são visões locais de trabalho; context/doctor ajudam a diagnosticar relações. Use apenas quando houver múltiplos repos/consumidores com uma necessidade concreta e confira a documentação da versão. Um app Android único normalmente pode manter openspec/ no próprio repo.

view oferece exploração visual/interativa; completion pode facilitar comandos; feedback envia uma issue e envolve publicação externa. Estes recursos não precisam integrar seu ciclo diário. Não execute comandos de remoção de stores/worksets como rotina de limpeza sem avaliar os dados afetados.

29. Checklist diário e definição de pronto

Antes de propor

- Confirmar o app, branch, trabalho local e capabilities existentes.
- Ler o contexto vigente e limitar uma entrega vertical útil.
- Resolver apenas ambiguidades que impedem decidir comportamento ou verificar a entrega.

Antes de Apply

- Aceite observável; requisitos e tela coerentes; falha/recuperação definidos.
- Design proporcional a dados, lifecycle, segurança e compatibilidade afetados.
- Tasks com skills pertinentes e verificações; interfaces TDD acordadas quando for usar tdd.

Durante implementação

- Um resultado coerente por vez, com teste que observe o contrato.
- Só carregar os helpers pertinentes; preservar decisões e mudanças alheias.
- Separar defeito de implementação de nova decisão de produto.

Antes da entrega

- Evidência do snapshot correto, variante e ambiente; ausência de check marcada explicitamente.
- Gates obrigatórios resolvidos e revisão read-only com cobertura declarada.
- UI/segurança avaliadas no escopo que o produto exige; achados tratados ou risco aceito pela autoridade responsável.
- Deltas sincronizados, documentos reconciliados e diff final revisto.
- Archive, commit/PR/merge e release conduzidos como ações distintas com suas regras.

Sinais de desperdício ou falsa confiança

Entrevistar de novo sobre decisões aceitas; criar spec e tasks concorrentes no tracker; adicionar Koin/WorkManager para usar uma skill; marcar testes indisponíveis como feitos; tratar rotação como morte do processo; acreditar que um mock prova a fronteira que substituiu; apagar um teste para ficar verde; arquivar uma intenção não entregue como comportamento vigente.

O mínimo eficiente

Um contrato claro, uma pequena change, um owner por resultado, skills pertinentes, evidência suficiente e uma próxima ação explícita. O mesmo processo serve a apps diferentes; políticas de produto, dados, APIs, arquitetura e critérios de release precisam ser definidos para cada um.

30. Fontes, revisões e alcance deste tutorial

Documentação consultada em 6 de novembro de 2026. Os prompts, políticas de integração e exemplos Notas de Bolso são uma proposta didática original para este conjunto de skills, não um workflow oficial conjunto publicado pelos mantenedores.

Fonte	O que foi conferido
OpenSpec v1.16.0	Release e requisitos de instalação.
CLI da versão	Configuração, navegação, validação, instruções, archive e schema.
Workflows e ações	Comportamento das ações e distinção entre planejamento e código.
Integrações e nomes	Formas de invocação e nomes gerados por host.
Configuração/customização	Contexto, rules e operation guidance; orientação não executável.
Schema spec-driven	Artefatos, deltas, cenários e dependências.
Matt Pocock, revisão consultada	README e entrypoints engineering/productivity pertinentes.
TDD de Matt	Interfaces acordadas, observação de comportamento e red-green.
Setup de Matt	Tracker, domínio e alterações de configuração por repositório.
Skills CLI	Instalação/seleção, agente de destino e requisito Node.
Estado no Android	Distinção entre recriação de tela, morte do processo e dados persistidos.
Testes Android	Escolha de testes locais/instrumentados e fronteiras verificadas.
Seu repositório de skills	README, SKILL-ROUTING e as instruções locais atuais dos owners/helpers.

O snapshot Git publicado usado como base local é 49d2de5d4f373b5e4d1ead2a2dd72f55902e9c8f, com alterações posteriores de atribuição e Peer Review 1.9.0 ainda locais na preparação deste PDF. Consulte os arquivos instalados para saber o que realmente está disponível no app.

Limite da verificação

O histórico e o texto/gerador da edição anterior foram revistos. As fontes foram lidas; versão, opções selecionadas e nomes de skills OpenSpec foram conferidos no pacote instalado. O YAML foi analisado, a sintaxe de 10 blocos shell foi conferida e o delta de exemplo passou validate --strict em um scaffold temporário. O PDF foi renderizado e inspecionado. O app não foi implementado, os comandos Gradle não foram executados e a compatibilidade de todas as combinações de modelos não foi medida.

Um bom roteiro melhora clareza e rastreabilidade. Qualidade, segurança, acessibilidade e prontidão de release dependem da evidência do aplicativo real no ambiente e snapshot relevantes.